

How Nubank Built an In-house Logging Platform for 1 Trillion Log Entries



BYTEBYTEGO

OCT 27, 2025



184



1



12

Share

[**Crane Worldwide Logistics Joins Streamfest 2025 — Hear Their Story \(Sponsored\)**](#)

Come hear Crane Worldwide Logistics at Streamfest 2025

Featuring Jared Noynaert, VP of Engineering,
Crane Worldwide Logistics



Redpanda

NOVEMBER 5-6
STREAMFEST '25



Join us **November 5-6** for Redpanda Streamfest, the two-day online event exploring future of streaming data and real-time AI. Jared Noynaert, VP of Engineering at Crane Worldwide Logistics, will share insights on the fundamentals of modern data infrastructure — from isolation and auto-scaling to branching and serverless mode that redefine how organizations scale. It's a forward-looking look at the principles

shaping next-generation architectures and where Redpanda and the broader Kafka ecosystem fit in. Streamfest features keynotes, demos, and case studies from engineers building the systems of tomorrow.

Disclaimer: The details in this post have been derived from the details shared online by the Nubank Engineering Team. All credit for the technical details goes to the Nubank Engineering Team. The links to the original articles and sources are present in the references section at the end of the post. We've attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to address them.

When a company grows rapidly, its technical systems are often pushed to their limits.

This is exactly what happened with Nubank, one of the largest digital banks in the world. As the company scaled its operations, the logging platform it relied on began to show serious cracks. Logging may sound like a small part of a tech stack, but it plays a critical role. Every time a service runs, it generates logs that help engineers understand what is happening behind the scenes. These logs are essential for monitoring systems, fixing problems quickly, and keeping the platform stable.

Nubank was using an external vendor to handle its log ingestion and storage. Over time, this setup became both costly and difficult to manage. The engineering team had very limited visibility into how logs were collected and stored, which made troubleshooting production issues harder. If something went wrong, they could not always trust that the logs they needed would be available.

Costs were also rising fast, with no clear way to predict future spending. The only way to handle more load was to pay more money. On top of that, important tools like alerts and dashboards were tied directly to the vendor's platform, making it difficult to change providers. Finally, whenever log ingestion spiked, it would slow down the querying process, affecting incident response.

These challenges pushed Nubank to build its own in-house logging platform to take back control, reduce costs, and ensure a stable foundation for its fast-growing system.

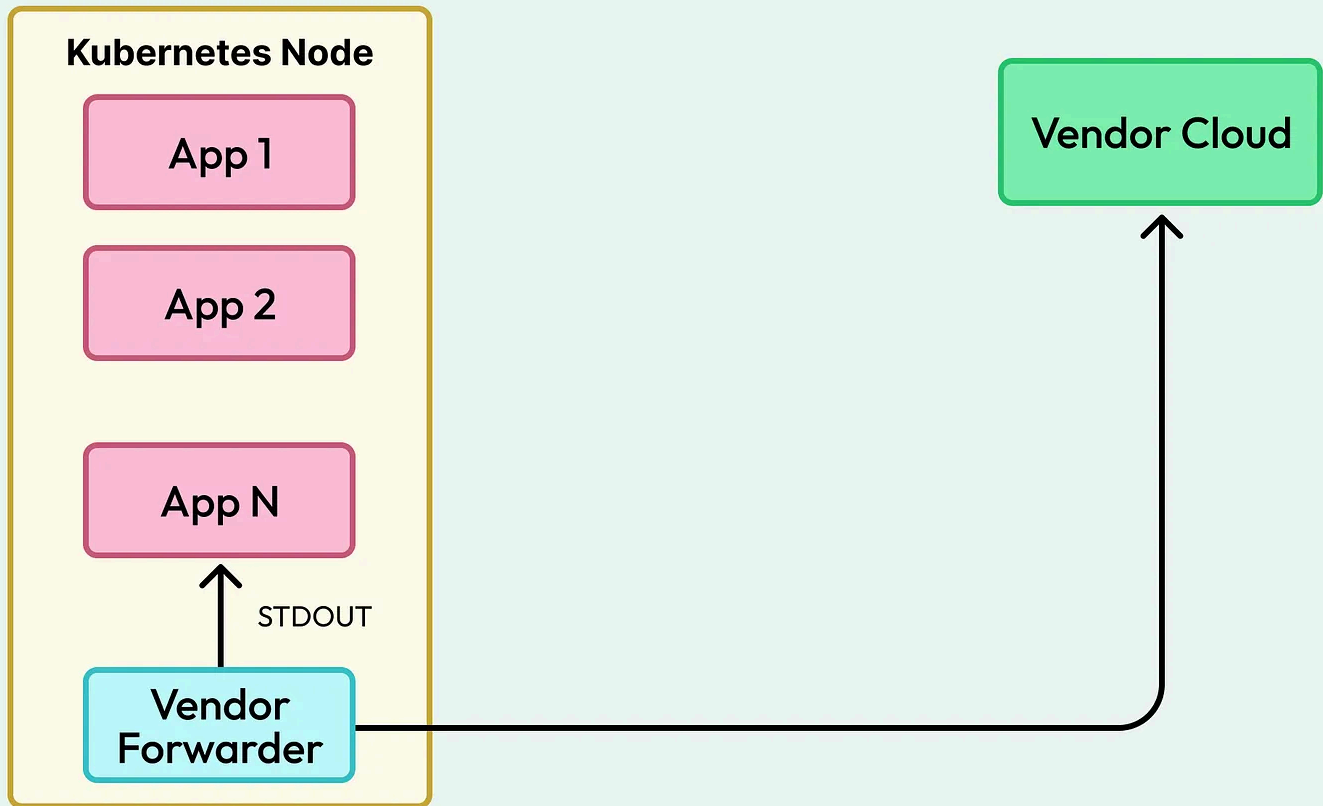
In this article, we will look at how Nubank built its in-house logging platform and the challenges the team faced.

The Initial Logging Architecture

Before building its own logging platform, Nubank relied entirely on a third-party vendor.

The setup was straightforward. Every application inside the company sends its logs directly to the vendor's forwarder or API. This worked well in the early days because the number of services was smaller, the log volume was manageable, and the system was simple to maintain.

Initial Log Infrastructure at Nubank



However, as the company grew, this basic setup began to fall apart due to the following reasons:

- The first problem was the lack of control over incoming data. Logs were sent as they were generated, with no way to filter out unnecessary information or route different kinds of logs to different places. This meant the system was processing a lot of data that didn't always add value, driving up costs.
- Another major issue was limited visibility into the logging pipeline. If something failed inside the vendor's platform, the engineering team had little insight into where or why it happened. These "blind spots" made it difficult to detect problems early or trust that logs were being handled correctly.
- The costs were also growing quickly. As more applications were added, the volume of logs increased, and so did the bills. The system offered no easy way to

optimize or reduce those costs.

- Finally, the architecture itself was rigid. Because everything was tightly tied to vendor, Nubank could not easily change how logs were ingested, stored, or queried.

Two-Phase Platform Strategy

To solve its growing challenges, the Nubank engineering team decided not to build everything at once. Instead, they split the project into two clear phases.

The first phase was called the Observability Stream. This part focused on ingesting and processing logs. In other words, this involved collecting data from thousands of applications and preparing it for storage. The team wanted a platform that could handle large amounts of incoming data reliably and efficiently. It also needed to be flexible enough to apply filters, process different data types, and generate useful metrics.

The second phase was the Query and Storage Platform. Once logs were collected and processed, they needed to be stored in a way that made them easy to search. Engineers use logs daily to investigate incidents, debug issues, and understand system behavior. This platform had to support fast queries, store data at low cost, and remain scalable as Nubank's infrastructure kept expanding.

This two-phased approach made the migration smoother and allowed them to test and improve each part before moving on. The goal was to create a complete, in-house logging management system that could grow with Nubank's massive scale.

Both phases were designed with three key goals in mind:

- **Reliability:** The platform had to keep working smoothly even during spikes in usage or unexpected failures.

- **Scalability:** It needed to handle both short-term traffic bursts and long-term growth.
- **Cost efficiency:** The solution had to be significantly cheaper than commercial vendors while providing full control and visibility.

With these guiding principles, Nubank began building the new logging platform from the ground up.

Ingestion Pipeline

The Nubank engineering team began by focusing on the first phase of their new platform: the ingestion pipeline, also known as the Observability Stream.

This part of the system is responsible for collecting logs from many different applications, processing them in a structured way, and preparing them for storage. In their old setup, ingestion and querying were tightly connected, which meant a problem in one area could affect the other. By separating the two, Nubank gained the ability to scale and manage each part independently.

To build this pipeline, the team combined reliable open-source tools with in-house services:

- **Fluent Bit (Open Source):** Nubank chose Fluent Bit as the log forwarder because it is lightweight, easy to configure, and supported by the Cloud Native Computing Foundation (CNCF). It collects logs efficiently from different services and sends them into the pipeline.
- **Data Buffer Service (In-house):** This service takes in the logs from Fluent Bit and stores them temporarily in micro-batches. Instead of sending every single log entry by one, the service groups them into manageable chunks. This helps smooth out traffic spikes, improves reliability, and ensures the rest of the system does not get overwhelmed during peak load.
- **Filter & Process Service (In-house):** This is a highly scalable processing layer that can filter, enrich, or transform incoming log data before it moves downstream. It also collects real-time metrics about the ingestion flow, giving the team better

visibility into the health of the system. The service is designed to be extensible, new processing logic or filters can be added quickly when needed.

By introducing these components, Nubank improved the reliability of its ingestion pipeline.

Logs are now buffered and processed in a controlled manner, which makes the system more stable during sudden traffic increases. Ingestion is fully decoupled from querying, so problems in one layer do not bring down the other. The team also gained operational telemetry in the form of insightful metrics that help make better decisions about storage design and system scaling.

This solid foundation paved the way for the next phase of the project.

Query and Storage Platform

Once the ingestion pipeline was stable, the Nubank engineering team turned its attention to the second major part of the platform: the query and storage layer.

This layer is what allows engineers to search through massive amounts of log data quickly and reliably. Since Nubank ingests trillions of logs every day, this part of the system needed to be both powerful and cost-efficient. It had to store data at a petabyte scale while still delivering fast query performance for daily debugging and monitoring.

Query Engine: Trino

For the query engine, the team chose Trino.

Trino is a distributed SQL engine that can query data stored in different locations. One of the main reasons for this choice was its partitioning capabilities.

Partitioning breaks data into smaller, structured pieces so that a query only scans the parts it needs. This improves performance and reduces the amount of computing

resources required.

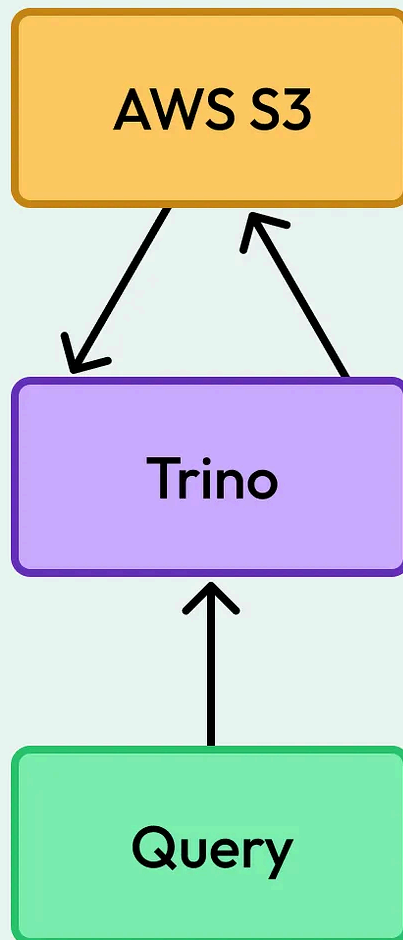
Trino also integrates well with multiple backends, which gives the platform flexibility to evolve in the future if needed.

Storage Layer: AWS S3

For storage, the team selected AWS S3.

This decision was driven by its high durability, strong availability guarantees, and practically infinite scalability. S3 can easily handle petabytes of data without the team needing to manage complex infrastructure. It is also cost-effective for long-term storage, which is essential when dealing with logs at this scale.

| S3 as Storage



As Nubank continues to grow, the storage layer can expand smoothly without major architectural changes.

Data Format: Parquet

The logs are stored in Parquet, a columnar data format designed for efficient querying.

Parquet provides an average 95 percent compaction rate, which drastically reduces how much space the data takes up. It also offers excellent scan performance, making

analytical queries much faster. In addition, Parquet uses efficient compression and serialization methods that help lower storage and compute costs.

Parquet Generator (In-house)

To convert the ingested log batches into Parquet format, the team built a high-throughput Parquet generator.

This is an in-house service designed to efficiently handle large volumes of data. Since it is internally controlled, the team can optimize it for cost, add custom features, and adapt it to new workloads over time. It is both scalable and extensible, ensuring the system can support future growth.

Scalability and Performance Metrics

By mid-2024, the new logging platform at Nubank had reached an impressive level of scale and efficiency. What started as a response to rising costs and limited visibility had grown into a powerful, cloud-native observability system. The team designed it to handle massive amounts of data without losing speed, reliability, or control.

Here are some key numbers that show how far the platform has come:

- It ingests 1 trillion logs every day, capturing activity from thousands of services and applications.
- It processes about 1 petabyte of data per day, preparing it for storage and querying.
- It maintains a 45-day retention period, which means the system holds around 4 petabytes of searchable data at any time.
- Engineers run around 15,000 queries per day, scanning 150 petabytes of data daily to monitor systems, troubleshoot incidents, and analyze behavior.
- The platform operates at 50 percent lower cost compared to the previous vendor-based solution.

Conclusion

The new logging platform built by the Nubank engineering team shows how a company can rethink its infrastructure to match the pace of its growth. By moving away from a costly and inflexible third-party solution, Nubank created a modern, cloud-native observability stack tailored to its own needs.

The success of this platform comes down to a few key design choices.

- First, the team decoupled ingestion from querying, which allowed both parts of the system to scale independently.
- They introduced micro-batching and buffering in the ingestion pipeline to handle massive data spikes reliably.

- For storage, they combined Parquet for efficient data formatting and AWS S3 for scalable and cost-effective storage at the petabyte level.
- For querying, they adopted Trino, which provides fast and resource-efficient access to massive datasets.
- Finally, they built several in-house services to maintain full control over processing, optimize performance, and adapt the system over time.

The result is a platform that provides predictable costs, high scalability, and deep operational visibility. It enables engineers to troubleshoot issues faster, build new features with confidence, and prepare the company for future growth.

References:

- [How Nubank Built Its In-House Log Platform](#)

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing sponsorship@bytebytego.com.



184 Likes • 12 Restacks

Discussion about this post

Comments Restacks

00

Write a comment...



RustyLion07 Oct 28, 2025

First of all, great work and outcome from a cost savings standpoint.

What you built seems very specific to logs; I hope your engineering teams are not relying only on logs, as that will sound outdated 😊

Are they able to correlate logs to other signals, such as real user transactions across services, distributed trace and Metrics? Most importantly, is that context available/presented vs. manually digging through multiple tools?

I often see that observability is equated to a signal, such as logs, which limits the core capabilities of a true observability system.

From a business standpoint, for observing modern distributed architectures, cost savings-focused goals sound more like a defensive approach. Rather, it should be more about how we can enhance user experience, increase developer productivity, or decrease Mean Time To Detection/Resolution (MTTD/R)?

The value driven by such outcomes outweighs the annual cost of observability by simply avoiding an outage, recovering faster, saving hours/days of war rooms, or increasing market share by delivering a top-rated experience. The recent AWS outage is a great example of a situation where, given a choice, they would want to invest all in observing the infrastructure versus recovering days; the latter is way more expensive, not just in lost revenue but also in brand perception in a highly competitive market.

♡ LIKE (3) 💬 REPLY



© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture